LAB: Timer & PWM

Date: 2026-09-02

Author/Partner: Choi Junghyun

Demo Video: https://youtu.be/mTD0_Cfr7RY

Introduction

Create a simple program that control a sevo motor and a DC motor with PWM output.

You must submit

- LAB Report (*.md & *.pdf)
- Zip source files(main*.c, ecRCC2.h, ecGPIO2.h, ecSysTick2.c etc...).
    - Only the source files. Do not submit project files

Requirement

Hardware

> ⚠️ **Warning**
>
> In this lab, **do not use the JKIT Eval Board**.
> Please disconnect it before proceeding with the following lab steps.

- MCU
    - NUCLEO-F411RE
- Actuator/Sensor/Others:
    - RC Servo Motor (SG90)
    - DC motor (5V)
    - DC motor driver(LS9110s or L298N)

Software

- PlatformIO, CMSIS, EC_HAL library

Prelab: Timer Out Register

{% content-ref url="/pages/oRLfuWvsGARepVStqFRd" %} PreLAB: Timer Interrupt & PWM {% endcontent-ref %}

Declare and define the following functions in your library. You must update your header files located in the directory `EC \include\`.

TIMER Interrupt API

```
// Timer Update Interrupt Initialization
void TIM_UI_init(TIM_TypeDef* TIMx, uint32_t msec);     // 16-bit  msec < 65,530
msec
void TIM_UI_init_us(TIM_TypeDef* TIMx, uint32_t usec);  // 16-bit  usec < 65,530
usec

// Timer Update Interrupt On/OFf
void TIM_UI_enable(TIM_TypeDef* TIMx);
void TIM_UI_disable(TIM_TypeDef* TIMx);

// Timer Update Interrupt Flag
uint32_t is_UIF(TIM_TypeDef *TIMx);
void clear_UIF(TIM_TypeDef *TIMx);
```

PWM API

{% code expandable="true" %}

```
//  Low Frequency PWM initialization in msec. (below 1kHz)
void PWM_init(PinName_t pinName, uint32_t msec);    // 1 to 6,553 msec (16-bit)
//  High-Frequency PWM initialization in usec (1kHz to 100 kHz)
void PWM_init_us(PinName_t pinName, uint32_t usec); // 10 usec and up
// Duty ratio 0~1.0
void PWM_duty(PinName_t pinName, float duty);
// Set Pulse width in msec
void PWM_pulsewidth(PinName_t pinName, uint32_t pulse_width_ms);
// Set Pulse width in usec
void PWM_pulsewidth_us(PinName_t pinName, uint32_t pulse_width_us);
```

{% endcode %}

Tutorial: DC Motor Driver

Learn how to connect a DC motor to the LS9110s motor driver

{% embed url="https://ykkim.gitbook.io/ec/ec-course/tutorial/tutorial-dcmotor-motor-driver-connection" %}

Problem 0: STM-Arduino

We will create a simple program to control a DC motor using a PWM input signal.

- Connect the DC motor to the DC motor driver
- Pressing the user button will turn on the DC motor

Procedure

1. Create a new project under the directory `lab\`

2. Open *Arduino IDE* and Create a new program named as '**TU_arduino_PWM.ino**'.
3. Write the following code.
4. Upload and Run

{% code expandable="true" %}

```
const int pwmPin = 11;   // PWM pin
const int buttonPin = 3;  // button pin

int buttonState = HIGH;

void setup() {
  pinMode(pwmPin, OUTPUT);

 // initialize the pushbutton pin as an input:
  pinMode(buttonPin, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(buttonPin), motorOperation, CHANGE);
}

void loop() {

  if (buttonState == LOW){
    for (int i = 0; i < 10; i++){
      analogWrite(pwmPin, 100 + 10*i);
      delay(100);
    }

    for (int i = 10; i > 0; i--){
      analogWrite(pwmPin, 100 + 10*i);
      delay(100);
    }
  }
  else{
    analogWrite(pwmPin, 0);
  }
}

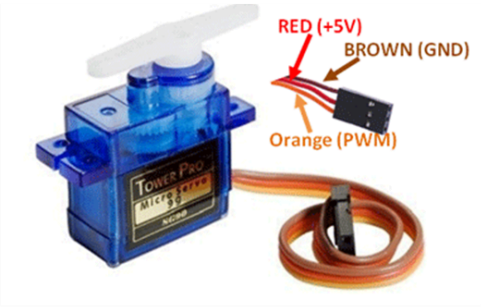
void motorOperation(){
  buttonState = digitalRead(buttonPin);
}
```

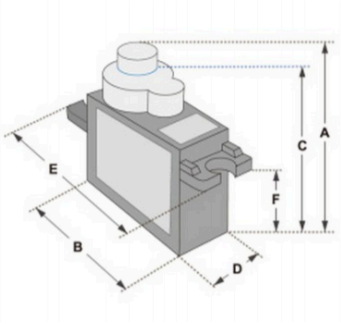
{% endcode %}

Problem 1: RC servo motor

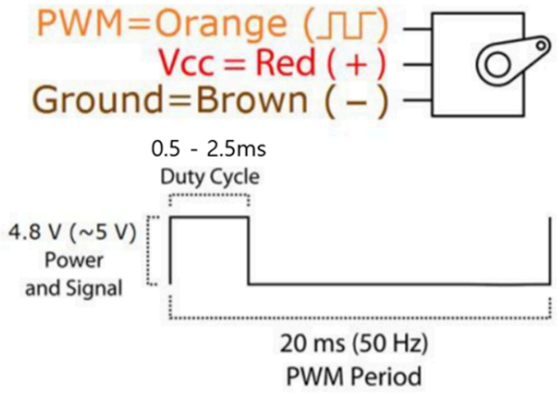
An RC servo motor is a lightweight motor with high output power. It is used to control rotation angles of approximately 180 degrees (90 degrees in each direction) and is commonly applied in RC cars and small-scale robots. The angle of the motor can be controlled by the pulse width (duty ratio) of a PWM signal. The PWM

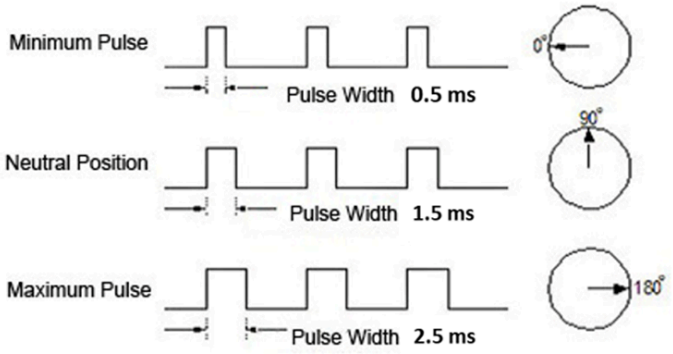
period should be set to 20ms (or 50Hz). Refer to the datasheet of the RC servo motor for detailed specifications





Dimensions & Specifications
A (mm) : 32
B (mm) : 23
C (mm) : 28.5
D (mm) : 12
E (mm) : 32
F (mm) : 19.5
Speed (sec) : 0.1
Torque (kg-cm) : 2.5
Weight (g) : 14.7
Voltage : 4.8 - 6





Make a simple program that changes the angle of the RC servo motor that rotates back and forth from 0 deg to 180 degree within a given period of time.

- Connect the RC servo motor to MCU pin (PA15) , VCC and GND
  - Use Port A Pin 15 as PWM output pin for TIM2_CH1.
  - Use Timer interrupt of period 500msec.
- Increase the angle of RC servo motor from 0° to 180° with a step of 10° every 500msec.
- After reaching 180°, decrease the angle back to 0°.
  - Use timer interrupt IRQ.
- When the user button (PA_4) is pressed, it should reset to the angle 0° and start over.
  - Use EXT interrupt.

> You need to observe how the PWM signal output is generated as the input button is pushed, using an oscilloscope. You need to capture the Oscilloscope output in the report.

Procedure

1. Create a new project under the directory `EC\lab\`

- Enviornment: "**LAB_PWM_RC".**
- Source File: "**LAB_PWM_RC.c"**

> You MUST write your name on the source file inside the comment section.

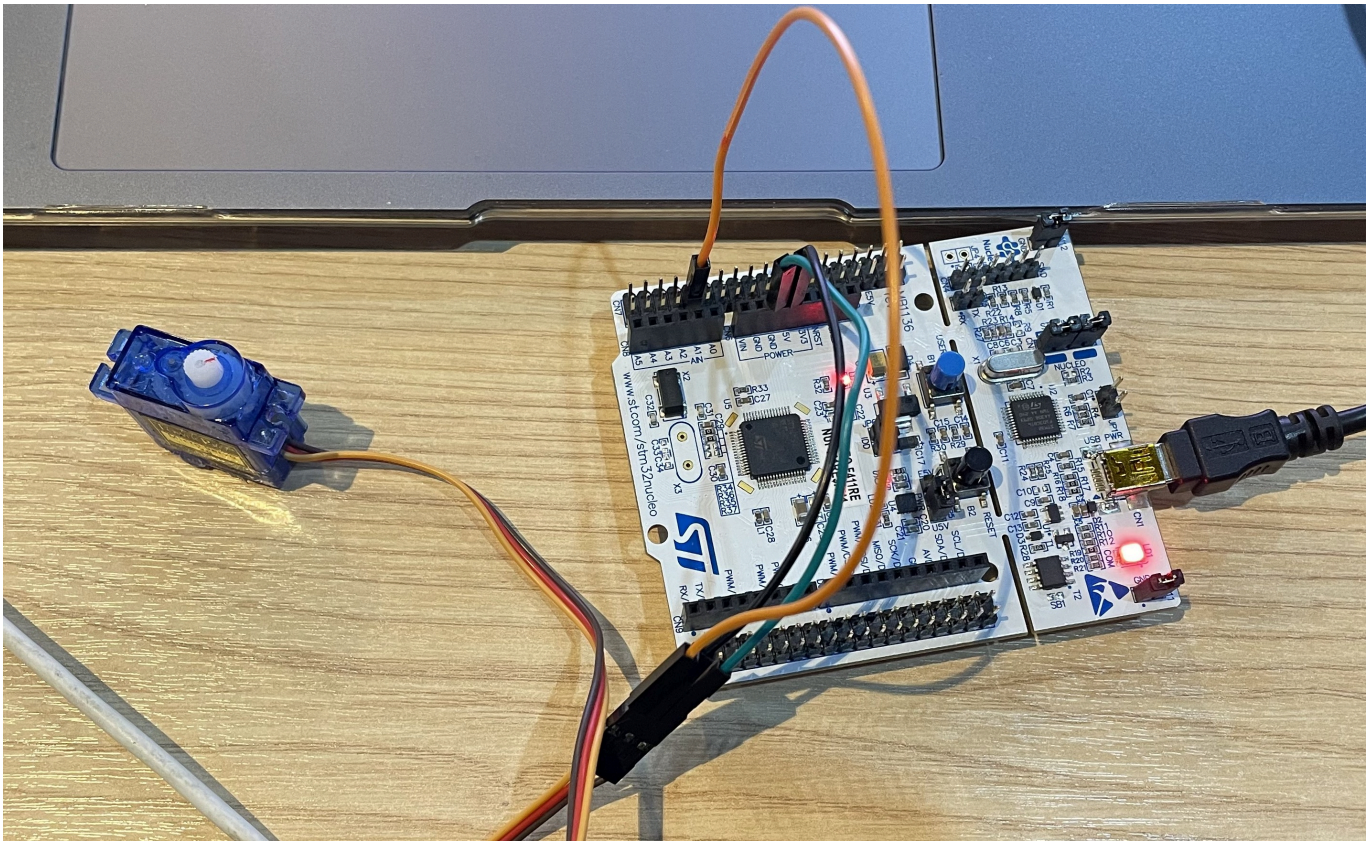
2. You must modify the `platformio.ini` **,** to add new environment.

Configuration

Type	Port - Pin	Configuration
Button	(PC13)	DIN, Pull-Up
PWM	TIM2_CH2 (PA1)	TIM2 (PWM) period: 20msec, Duty : 0.5~2.5msec
Timer Update Interrupt	TIM3	TIM3: Timer Interrupt of 500 msec

Circuit Diagram

> You need to include the circuit diagram (if necessary)



Code

Your code goes here:

Explain your source code with necessary comments.

```
/*-----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : [Choi Junghyun]
Created          : 11-02-2026
Modified         : 11-07-2026
Language/ver     : C++ in VS Code
Description      : RC motor 10degree angle control, lab problem1
/-----------------------------------------------------------------*/
```

```c
#include "stm32f4xx.h"
#include "ecRCC2.h"
#include "ecGPIO2.h"
#include "ecPWM2.h"
#include "ecTIM2.h"
#include "ecEXTI2.h"

#define PWM_PIN     PA_1
#define BUTTON_PIN  PC_13

volatile int angle = 0;
volatile int direction = 1;

void setup(void);
void TIM3_IRQHandler(void);
void EXTI15_10_IRQHandler(void);

int main(void) {
    setup();

    while (1) {
        // All control logic handled by interrupts
    }
}

void setup(void) {

    // System clock: 84 MHz
    RCC_PLL_init();

    // Servo mototr PWM: TIM2_CH2, 20ms period
    PWM_init(PA_1, 20);
    PWM_pulsewidth_us(PWM_PIN, 500);

    // TIM3 interrupt: 500ms
    TIM_UI_init(TIM3, 500);
    TIM_UI_enable(TIM3);

    NVIC_SetPriority(TIM3_IRQn, 1);
    NVIC_EnableIRQ(TIM3_IRQn);

    // button PC_13 setting
    GPIO_init(BUTTON_PIN, INPUT);
    GPIO_pupd(BUTTON_PIN, EC_PU);

    EXTI_init(BUTTON_PIN, FALL, 0);

    NVIC_SetPriority(EXTI15_10_IRQn, 0);
    NVIC_EnableIRQ(EXTI15_10_IRQn);
}


// TIM3: Every 500ms
void TIM3_IRQHandler(void) {
```

```
    if (is_UIF(TIM3)) {

        clear_UIF(TIM3);

        // angle change by 10 degrees
        angle += direction * 10;

        // Reverse direction at boundaries
        if (angle >= 180) {
            angle = 180;
            direction = -1;
        }

        else if (angle <= 0) {
            angle = 0;
            direction = 1;
        }

        // Convert angle to pulse width(500~2500us)
        uint32_t pulse_width = 500 + (angle * 2000) / 180;

        PWM_pulsewidth_us(PWM_PIN, pulse_width);
    }
}


// EXTI: Button pressed
void EXTI15_10_IRQHandler(void) {

    if (is_pending_EXTI(BUTTON_PIN)) {

        // Clear interrupt flag
        clear_pending_EXTI(BUTTON_PIN);

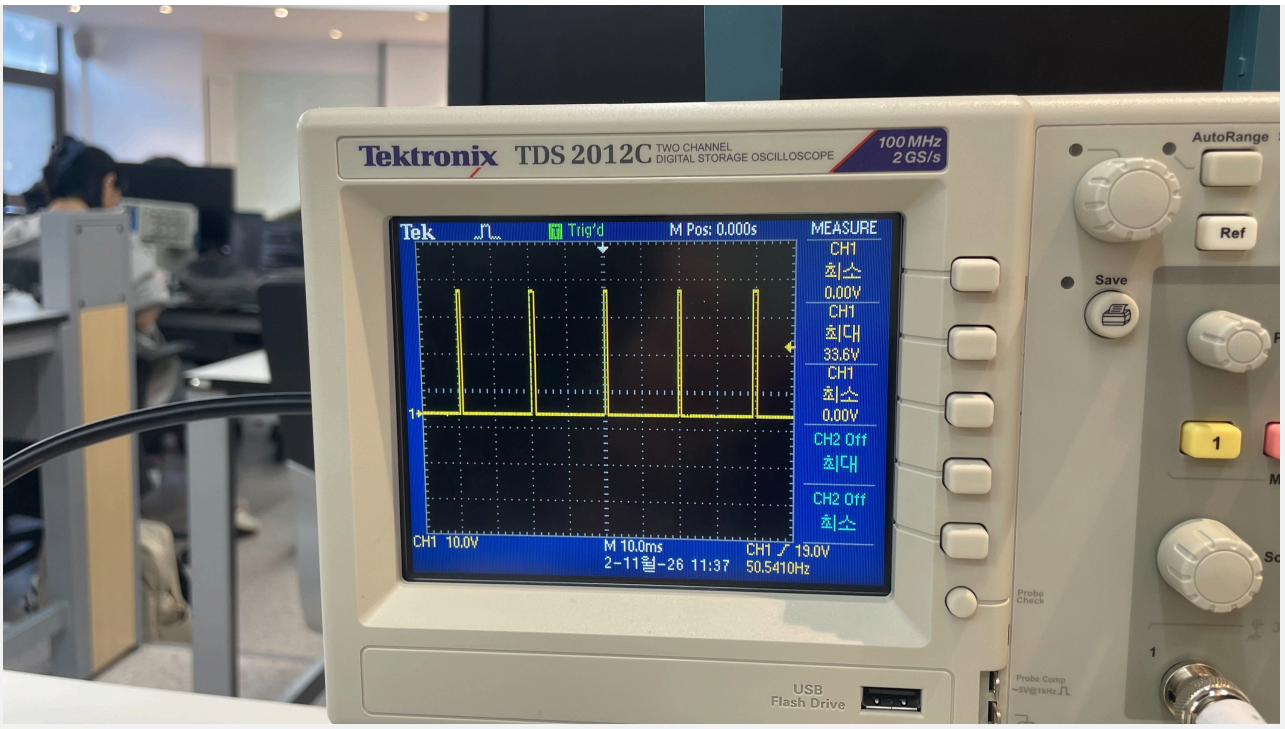
        // Reset motor state
        angle = 0;
        direction = 1;

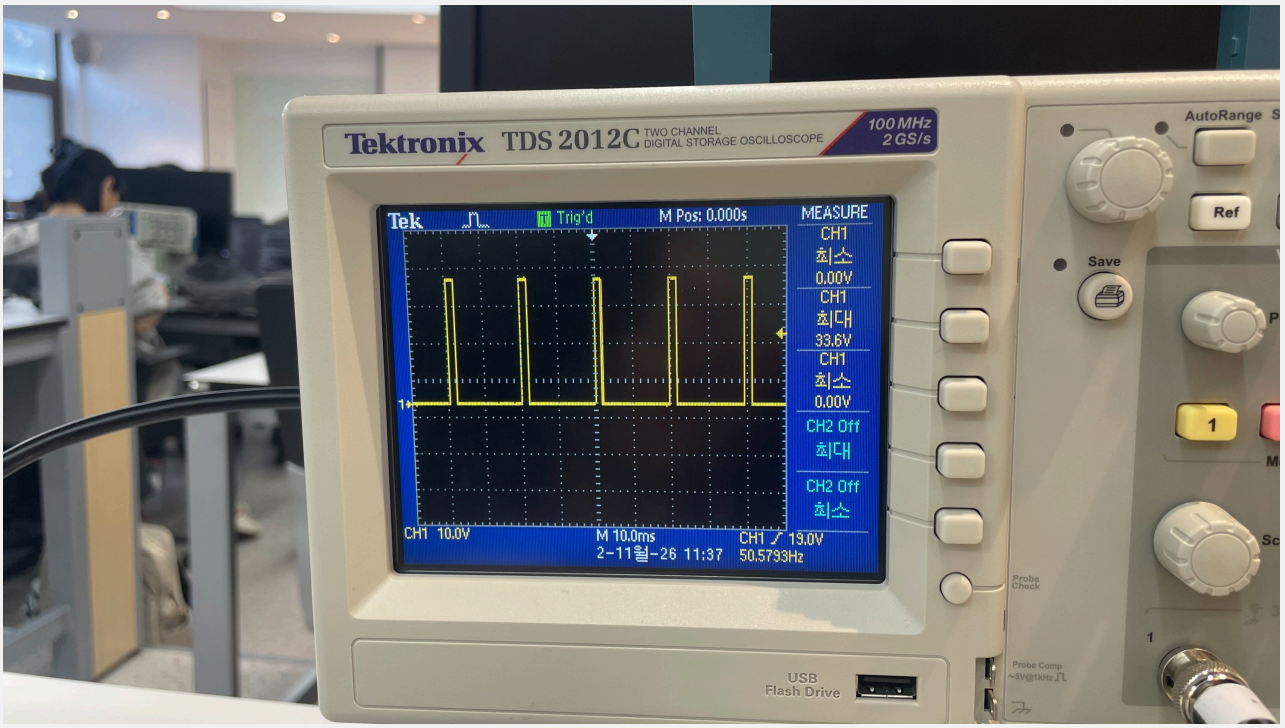
        // Move motor to 0 degrees
        PWM_pulsewidth_us(PWM_PIN, 500);
    }
}
```

Results

Experiment images and results

> Show experiment images /results first image is the oscilloscope at 0 degrees, second image is at 90 degrees





Add demo video link

Discussion

1. Derive a simple logic to calculate CRR and ARR values to generate x[Hz] and y[%] duty ratio of PWM. How can you read the values of input clock frequency and PSC?

> To derive the ARR and CCR for a PWM signal with frequency and duty ratio, start by calculating the PWM period $T_{\text{PWM}} = \frac{1}{x}$. The timer's counting clock frequency depends on the timer input clock (f_{CLK}) divided by the prescaler plus one ($f_{\text{timer}} = \frac{f_{\text{CLK}}}{\text{PSC}+1}$), giving a single tick duration of $T_{\text{tick}} = \frac{\text{PSC}+1}{f_{\text{CLK}}}$. The total number of timer counts required for one full PWM period determines the ARR value, calculated as $\text{ARR} = \left(\frac{f_{\text{CLK}}}{x \cdot (\text{PSC}+1)}\right) - 1$. Once ARR is set, the pulse high-duration counter value CCR is derived directly from the percentage duty ratio using $\text{CCR} = (\text{ARR}+1) \cdot \frac{y}{100}$. In STM32 bare-

> metal programming, is is possible to read the input clock frequency and prescaler dynamically from system and hardware registers. the timer prescaler value is read directly from the timer's prescaler register (TIMx->PSC), while the timer clock frequency f_{CLK} is obtained either via CMSIS RCC system clock functions like SystemCoreClock (scaled by APB prescalers from RCC registers) or by retrieving the configured clock rate from your course library initialization header (ecRCC2.h).

2. What is the smallest and highest PWM frequency that can be generated for Q1?

> For TIM2 operating on an 84 MHz timer clock on the STM32F4 series, the maximum and minimum achievable PWM frequencies are based on the limits of its 16-bit Prescaler (PSC) and Auto-Reload Register (ARR). The lowest possible PWM frequency is achieved by maximizing both registers to their 16-bit capacity ($\mathrm{PSC} = 65535$ and $\mathrm{ARR} = 65535$), which yields $f_{\mathrm{PWM,\,min}} = \frac{84{,}000{,}000}{(65535+1)\cdot(65535+1)} \approx 0.01956\ \mathrm{Hz}$ (a period of approximately $51.12\ \mathrm{seconds}$). The highest theoretical PWM frequency occurs when setting both registers to their minimum possible values ($\mathrm{PSC} = 0$ and $\mathrm{ARR} = 1$), resulting in $f_{\mathrm{PWM,\,max}} = \frac{84{,}000{,}000}{(0+1)\cdot(1+1)} = 42\ \mathrm{MHz}$; however, because $\mathrm{ARR} = 1$ limits the duty cycle resolution to only two steps (0% or 100%), practical applications requiring fine duty cycle control (e.g., 1% steps) typically cap the maximum operational frequency at $840\ \mathrm{kHz}$.

Problem 2: DC motor

> **you MUST read** Tutorial: DC motor driver connection

Problem

The program needs to design a smart mini-fan device that runs only when the face is near the fan. Otherwise stops the fan from running.

- As the button **B1** is pressed, change the fan velocity Mode.
- The velocity MODE(states) are
  - MODE(state): **OFF(0%), MID(50%), HIGH(100%)**
- PAUSE: When the object(face) is detected neraby, it automatically pauses the fan temporarily.
  - PAUSE: the MODE should NOT be changed if the button **B1** is pressed
- RESUME: When the object(face) is away, it automatically resumes the fan running
  - It must maintain return to the previous MODE run at the speed of the current MODE
- LED
  - MODE=OFF, LED=OFF
  - MODE=MID, HIGH, LED=ON
  - PAUSE, LED=Blinking 1 sec
- Serial Display
  - Print the Mode, PWM duty ratio and other necessary information every 2seconds
- Use FSM to design the code
  - Draw a FSM(finite-state-machine) table and state diagram
  - Example Table. See below for example codes

Present state	Next state (X, Y)				Output Z			
	(0, F)	(0, T)	(1, F)	(1, T)	(0, F)	(0, T)	(1, F)	(1, T)
S0	S0	S0	S1	S1	V=0 L=0	V=0 L=0	V=0 L=0	V=50, L=1
S1	S1	S1	S2	S2	V=0, L=B	V=50, L=1	V=0, L=B	V=100, L=1
S2	S2	S2	S0	S0	V=0, L=B	V=100, L=1	V=0, L=B	V=0, L=0

Procedure

1. Create a new environment

- Environment: "**LAB_PWM_DC"**
- Source FIle: "**LAB_PWM_DC.c"**

> You MUST write your name on the source file inside the comment section.

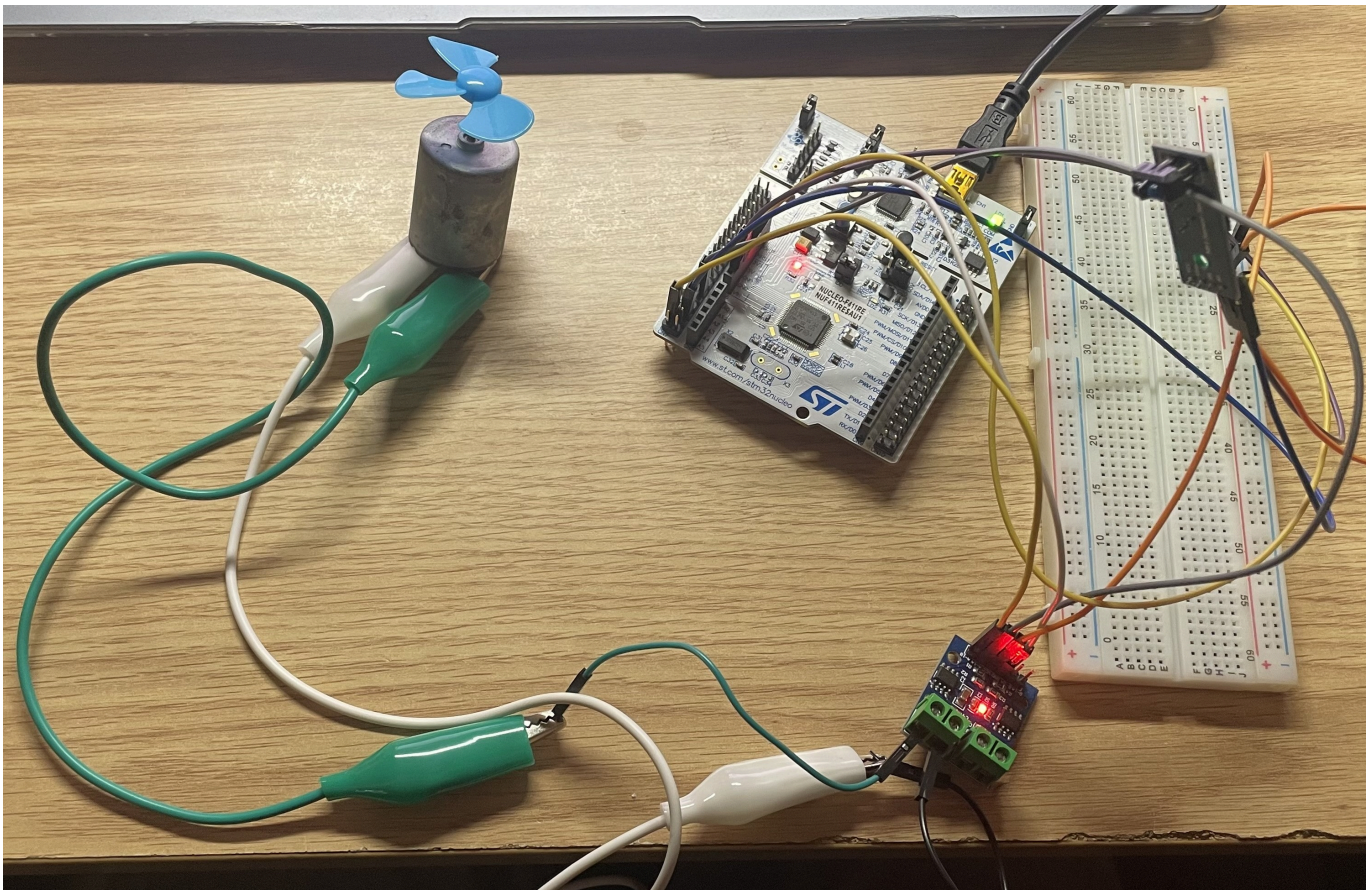
2. You must modify the `platformio.ini` **,** to add new environment.

Configuration

Function	Port - Pin	Configuration
Button	PC13	DIN, Pull-Up
Direction Pin	PC2	DOUT, Push-Pull
PWM Timer	TIM2_CH1 (PA_0)	TIM2 (PWM) period: **1msec (1kHz)**
Timer Update Interrupt	TIM3	TIM3: Timer Interrupt of 500 msec
LED		
IR Proximity Sensor		DIN, Pull-Up

Circuit Diagram

> You need to include the circuit diagram



Code

Your code goes here:

Explain your source code with necessary comments.

```
/*----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : Choi Junghyun
Created          : 11-07-2026
Modified         : 11-07-2026
Language/ver     : C / C++ in VS Code
Description      : DC motor fan control (Lab Problem 2 - Smart Mini-Fan)
/----------------------------------------------------------------*/

#include "stm32f4xx.h"
#include "ecRCC2.h"
#include "ecGPIO2.h"
#include "ecPWM2.h"
#include "ecTIM2.h"
#include "ecEXTI2.h"
#include "ecUART2.h"
#include <stdio.h>

#define PWM_PIN       PA_0       // TIM2_CH1
#define DIR_PIN       PC_2       // motor direction
#define BUTTON_PIN    PC_13      // speed button B1
#define LED_PIN       PA_5       // Onboard LED
```

```
#define IR_PIN        PB_0       // IR sensor

#define IR_DETECTED_LEVEL  0     // LOW(0) = Object detected (Face nearby)

typedef enum {
    FAN_OFF,
    FAN_MID,
    FAN_HIGH,
    FAN_PAUSE
} FanState_t;

volatile FanState_t fanState = FAN_OFF;
volatile FanState_t previousState = FAN_OFF; // Stores active mode before stopping

volatile uint8_t button_event = 0;
volatile uint8_t button_lockout = 0;

volatile uint8_t blink_event = 0;
volatile uint8_t serial_event = 0;

volatile int blink_counter = 0;
volatile int serial_counter = 0;

uint8_t led_state = 0;
float currentDuty = 0.0f;

// retarget standard C stdout printf to UART2 for Terra Term for display
int __io_putchar(int ch) {
    USART2_write((uint8_t*)&ch, 1);
    return ch;
}


void setup(void);
void applyOutputs(void);
void changeMode(void);
const char* getStateName(FanState_t state);

void TIM3_IRQHandler(void);
void EXTI15_10_IRQHandler(void);


int main(void) {
    setup();

    while (1) {
        // check IR sensor
        int objectDetected = (GPIO_read(IR_PIN) == IR_DETECTED_LEVEL);

        // Object detected -> Transition to PAUSE
        if (objectDetected && fanState != FAN_PAUSE) {
            previousState = fanState;       // Save active running state
            fanState = FAN_PAUSE;
```

```
            button_event = 0;               // Clear pending button events
            led_state = 0;
            GPIO_write(LED_PIN, 0);

            applyOutputs();
        }
        // Object away -> RESUME previous mode
        else if (!objectDetected && fanState == FAN_PAUSE) {
            fanState = previousState;       // Restore saved state
            led_state = 0;

            applyOutputs();
        }

        // Button event
        if (button_event) {
            button_event = 0;

            // Button B1 MUST NOT change mode during PAUSE
            if (fanState != FAN_PAUSE) {
                changeMode();
                applyOutputs();
            }
        }


        // LED blinking
        if (blink_event) {
            blink_event = 0;

            if (fanState == FAN_PAUSE) {
                led_state = !led_state;
                GPIO_write(LED_PIN, led_state);
            }
        }

        // terra form displayu
        static FanState_t lastPrintedState = (FanState_t)-1;

        if (serial_event) {
            serial_event = 0;

            if (fanState != FAN_OFF || fanState != lastPrintedState) {
                printf("State: %-5s | PWM Duty: %3.0f%% | IR Sensor: %s\r\n",
                   getStateName(fanState),
                   currentDuty * 100.0f,
                   (fanState == FAN_PAUSE) ? "DETECTED (PAUSED)" : "CLEAR");

                lastPrintedState = fanState; // Update last printed state
            }
        }
    }
}
```

```
void setup(void) {
    // System Clock
    RCC_PLL_init();

    // UART Initialization for Terra Term (9600 Baud)
    USART2_init();

    // GPIO Configuration
    GPIO_init(DIR_PIN, OUTPUT);
    GPIO_write(DIR_PIN, 0);// Set forward direction

    GPIO_init(LED_PIN, OUTPUT);
    GPIO_write(LED_PIN, 0);// Initial LED state OFF

    GPIO_init(IR_PIN,0);
    GPIO_pupd(IR_PIN, EC_PU);
    // Pull-up for IR proximity digital pin

    GPIO_init(BUTTON_PIN, 0);
    GPIO_pupd(BUTTON_PIN, EC_PU);
    // Pull-up for User Button PC13

    // PWM Configuration (1kHz Frequency on PA0)
    PWM_init(PWM_PIN, 1);// 1ms period = 1kHz
    PWM_duty(PWM_PIN, 0.0f);// Initial duty 0%

    //Timer Interrupt Setup(TIM3 - 500ms Update Period)
    TIM_UI_init(TIM3, 500);
    TIM_UI_enable(TIM3);
    NVIC_SetPriority(TIM3_IRQn, 1);
    NVIC_EnableIRQ(TIM3_IRQn);

    //External Interrupt Setup for PC13 Button
    EXTI_init(BUTTON_PIN, FALL, 0);
    NVIC_SetPriority(EXTI15_10_IRQn, 0);
    NVIC_EnableIRQ(EXTI15_10_IRQn);
}
// changing fan speed from 0-> mid -> max
void changeMode(void) {
    switch (fanState) {
        case FAN_OFF:
            fanState = FAN_MID;
            break;
        case FAN_MID:
            fanState = FAN_HIGH;
            break;
        case FAN_HIGH:
            fanState = FAN_OFF;
            break;
        default:
            break;
    }
}
```

```
void applyOutputs(void) {
    switch (fanState) {
        case FAN_OFF:
            currentDuty = 0.0f;
            PWM_duty(PWM_PIN, currentDuty);
            GPIO_write(LED_PIN, 0);
            break;

        case FAN_MID:
            currentDuty = 0.5f;
            PWM_duty(PWM_PIN, currentDuty);
            GPIO_write(LED_PIN, 1);
            break;

        case FAN_HIGH:
            currentDuty = 1.0f;
            PWM_duty(PWM_PIN, currentDuty);
            GPIO_write(LED_PIN, 1);
            break;

        case FAN_PAUSE:
            currentDuty = 0.0f;
            PWM_duty(PWM_PIN, currentDuty);
            // LED blinking is handled dynamically in main loop via blink_event
            break;
    }
}


// get state names for display

const char* getStateName(FanState_t state) {
    switch (state) {
        case FAN_OFF:   return "OFF";
        case FAN_MID:   return "MID";
        case FAN_HIGH:  return "HIGH";
        case FAN_PAUSE: return "PAUSE";
        default:        return "UNKNOWN";
    }
}

// TIM3 interrupt
void TIM3_IRQHandler(void) {
    if (is_UIF(TIM3)) {
        clear_UIF(TIM3);

        // Button debounce lockout counter
        if (button_lockout > 0) {
            button_lockout--;
        }

        // LED toggle flag every 1 second(2*500ms)
        blink_counter++;
```

```
            if (blink_counter >= 2) {
                blink_counter = 0;
                blink_event = 1;
            }

            //Serial print flag every 2 seconds (4*500ms)
            serial_counter++;
            if (serial_counter >= 4) {
                serial_counter = 0;
                serial_event = 1;
            }
        }
    }

    // EXTI interrupt handler
    void EXTI15_10_IRQHandler(void) {
        if (is_pending_EXTI(BUTTON_PIN)) {
            clear_pending_EXTI(BUTTON_PIN);

            // Debounce lock: Ignore additional edges within lockout window
            if (button_lockout == 0) {
                button_lockout = 1; // Locked out for 500ms

                // Ignore button presses during PAUSE state
                if (fanState != FAN_PAUSE) {
                    button_event = 1;
                }
            }
        }
    }
```

Results

Experiment images and results

> Show experiment images /results

Add demo video link

Reference

List of all references used (github, blog, paper, etc)

Troubleshooting

1. motor PWM duty ratio for different DIR

When, DIR=0 duty=0.8--> PWM 0.8 // 실제 모터에 전달되는 pwm

Whe, DIR=1 duty=0.8--> PWM 0.2 // 실제 모터에 전달되는 PWM

- Sample Solution

```
float targetPWM;  // pwm for motor input
float duty=abs(DIR-targetPWM); // duty with consideration of DIR=1 or 0

PWM_duty(PWM_PIN, duty);
```

2. Motor does not run under duty 0.5

Configure motor PWM period above 1kHz

3. Check and give different Interrupt Priority

Check if you have different NVIC priority number for each IRQs